

软件工程 2025 年期末复习知识点清单

第一章 软件工程概述

1. 软件的概念和特点

概念

软件(Software)是指在计算机系统的支持下, 能够完成特定功能与性能的**程序(Program)**、**数据(Data)**和**相关文档(Document)**

↑ 简记



特点

□ 逻辑性

✓ **逻辑产品**, 思维活动 (而非物理活动) 的结果、不会磨损和老化

□ 复杂性

✓ **规模大**: 代码行、模块、介入人员、进程、数据等数量非常大

✓ **运行复杂**: 状态很难追踪和复现

□ 设计开发

✓ 是**设计开发**而成的, 不是生产制造而成的

□ 缺陷的隐蔽性

✓ 缺陷**隐藏在逻辑代码**中, 不像硬件系统那样直观显现, 很难被人们所发现和排除

□ 易变性

✓ 需求**经常变**、难以把控, 影响软件的制品即开发过程

2. 软件危机的概念和产生的原因

概念: 软件危机是指在软件开发和维护的过程中, 由于需求的复杂性、技术的限制和管理的不完善, 导致生产成本过高、质量难以保证、无法按时完成开发的现象

原因:

□ 对软件这样一类**复杂和特殊系统**的认识不清

✓ 软件是新生事物, 对其特点、规律性和复杂性认识不够

□ 没有找到支持软件系统开发的**有效方法**

✓ 基础理论、关键技术、开发过程、支撑工具等

□ 缺乏成功软件开发**实践**以及相应的开发**经验**

✓ 系统总结、认真分析、充分借鉴、吸取教训

3. 软件工程的定义、软件工程的定义、概念、构成要素、目标和原则

定义/概念

将系统化、规范化、可量化的方法应用于软件的开发、运行和维护的过程；以及上述方法的研究 -- [IEEE 93]

要素

- **过程**
- **方法**
- **工具**

目标

在成本、进度等约束下，指导软件开发和运维，开发出满足用户要求的足够好软件

↑ 简记

满足约束和要求

指导软件开发

确保软件质量

原则

原则1：抽象和建模

原则2：模块化

原则3：软件重用

原则4：信息隐藏

原则5：关注点分离

原则6：分而治之

原则7：双向追踪原则

原则8：工具辅助

4. 开源软件的概念和应用、软件创作和软件生产

一种源代码可以**自由获取和传播**的计算机软件，其拥有者通过**开源许可证**赋予被许可人对软件**进行使用、修改和传播**

□ 操作系统

✓ Linux、Ubuntu、麒麟、鸿蒙、OpenEuler等

□ 数据库系统

✓ MySQL、PostgreSQL、MongoDB、Redis等

□ 开发平台

✓ Eclipse、Junit、SonarQube、Kubernetes等

□ 人工智能

✓ Tensorflow、Opencv、Caffe、Deeplearning4j等

□ 网络安全

✓ Nmap、curity Onion、Suricata、Bro等



第二章 软件过程

1. 软件生命周期概念

在软件工程中, 软件生命周期 (Software Life Cycle, SLC), 也称为软件开发生命周期 (SDLC), 是指软件产品从最初的构思、开发、运行、维护直到最后被废弃的整个过程。

2. 软件过程及模型概念

软件过程概念

□ 软件过程 (Software Process)

✓ 按照项目进度、成本和质量要求, 遵循用户需求, 开发和维护软件、管理软件项目的一系列有序软件开发活动

✓ 软件开发活动: 技术活动和管理活动

软件过程模型概念

□ 软件过程模型 (Software Process Model)

✓ 定义了软件开发的具体活动以及活动间的逻辑关系

3. 常见的几种软件过程模型: 瀑布、增量、原型、螺旋、迭代, 比较各自优缺点

模型名称	指导思想	关注点	适合软件	管理难度
瀑布模型	提供系统性指导	与软件生命周期相一致	需求变动不大、较为明确、可预先定义的应用	易
原型模型	以原型为媒介指导用户的需求导出和评价	需求获取、导出和确认	理解需求难以表述清楚、不易导出和获取的应用	易
增量模型	快速交付和并行开发	软件详细设计、编码和测试的增量式完成	需求变动不大、较为明确、可预先定义的应用	易
迭代模型	多次迭代，每次仅针对部分明确软件需求	分多次迭代来开发软件，每次仅关注部分需求	需求变动大、难以一次性说清楚的应用	中等
螺旋模型	集成迭代模型和原型模型，引入风险分析	软件计划制定和实施，软件风险管理，基于原型的迭代式开发	开发风险大，需求难以确定的应用	难

4. 敏捷开发方法 定义

□一种轻量级软件开发方法

✓相对于重量级的软件开发方法而言

□主张软件开发要以代码为中心，快速、轻巧和主动应对需求变化，持续、及时交付可运行的软件系统

✓轻便、轻巧

□提供了一组思想和策略，指导快速响应用户需求的变化，快速交付可运行的软件制品

思想

□强化可运行的软件，弱化软件文档

✓以可运行软件为中心来开展软件开发

□以适应变化为目的，提升应变能力

✓针对变化不断进行优化和调整任务、产品和计划等

□以人为本，让方法等服务于人

✓敏捷软件开发是面向人的而不是面向过程的，让方法、技术、工具、过程等来适应人，而不是让人来适应它们

原则

1. 尽早和持续地交付有价值的软件，以使用户满意
2. 即使到了软件开发后期，也欢迎用户需求的变化
3. 不断交付可运行软件系统，交付周期可以从几周到几个月
4. 在整个软件项目开发期间，用户和开发人员最好能每天一起工作
5. 由积极主动的人来承担项目开发，给他们提供所需环境和支持，信任他们的能力
6. 团队内部最有效的信息传递方式是面对面的交谈

7. 将**可运行软件**作为衡量软件开发进度的首要标准
8. 可持续性的开发，出资方、开发方和用户方应当保持**长期、恒定的开发速度**
9. 关注**优秀的技能和良好的设计**会增强敏捷性
10. 采用**简单化**的方法来完成任
11. 最好的架构、需求和设计出自于**自组织的团队**
12. 软件开发团队应定期就如何提高工作效率的问题进行**反思，并进行相应的调整**

具体敏捷开发方法

- ✓极限编程
- ✓测试驱动开发方法
- ✓Scrum方法
- ✓DevOps方法

第三章 需求分析

1. 需求分析的概念

□定义1(从软件利益相关方的角度): 软件系统的**利益相关方**对软件系统的**功能和质量**，以及软件运行环境、交付进度等方面提出的**期望和要求**

□定义2(软件本身的角度): **软件需求**是指软件用于解决现实世界问题时所表现出的**功能和性能等方面的要求**

类别	内涵	关注的利益相关方	示例
功能性需求	软件具有的功能、行为和服务	用户、客户、开发者群体、其他系统	- 分析和识别老人语音呼叫 - 分析异常状况
软件质量需求	内部质量需求	开发者群体	- 可维护性、可扩展性、可理解性、可重用性等
	外部质量需求	用户、客户、开发者群体、其他系统	- 界面操作要在1秒内响应 - 视频延迟不超过2秒
开发约束性需求	软件开发需满足的要求	客户、开发者群体、其他系统	- 要求在6个月内交付产品 - 软件运行在Android之上 - 采用Java语言来实现

软件需求特点

隐式、隐晦、多源、易变、价值不均、领域相关

2. 需求分析的过程：需求确认与需求变更

软件需求确认

✓站在**用户和客户**的角度，**确保软件需求的正确性**，通常采用需求评审 (Review)、原型确认等方式。

软件需求的变更管理

✓**多变性和易变性**引起的，明确哪些方面的需求发生了变化、反应在软件需求模型和文档的哪些部分、导致软件需求模型和文档的版本发生了什么样变化等

3. 需求确认的步骤：需求获取→需求提炼→需求描述→需求验证
4. 需求分析三类建模：用例模型、结构模型、行为模型。

5. 面向对象的需求分析过程中，三类模型的具体分析方法，UML 图类型有哪些？

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

6. 掌握用例图（包含、扩展等）、顺序图、活动图作法，根据需求生成分析类图（关联、依赖、聚合、组合……）。活动图要求能用泳道划分活动图、分支及汇合、分叉及合并

※本学期需要掌握六类图画法(用例图，顺序图，分析类图，活动图，包图，类图)

第四章 软件设计

1. 系统设计分为概要设计和详细设计
2. 与设计相关的 8 个概念：抽象、体系结构、设计模式、模块化、信息隐藏、功能独立、细化、重构。

抽象

抽象在软件设计中的应用

- ✓软件开发就是一个**从高层抽象到低层抽象逐步过渡过程**
- ✓**高层设计(架构设计) → 底层设计(详细设计)**，“逐步求精”过程

体系结构

软件体系结构(Software Architecture, SA)

- ✓也称软件架构，从**高层抽象**角度刻画组成软件系统的**设计元素**及它们之间的**逻辑关联**

设计模式

1.5 软件设计模式

- 借用以往的**经验来解决问题**
- 以**设计重用为目的**，采用一种**良定义的、正规的、一致的**方式记录的软件设计经验
- 关注在一般或特定设计环境中可能重复出现的设计问题，并给出经过**充分实践考验**的软件解决方案

□体系结构风格

- ✓面向整个软件系统

□构件设计模式

- ✓面向子系统或者构件

□实现设计模式

- ✓针对子系统或构件中的某个特定问题

模块

将软件系统的整体结构分解为一组相对独立的模块

- ✓模块：**包、子系统、构件、类、方法等等**
- ✓**每个模块实现单一的功能**
- ✓通过模块之间的交互来组装模块，形成整体框架
- ✓体现了“分而治之”思想

如何达成模块化

- ✓模块内部强内聚
- ✓模块之间低耦合



信息隐藏

何为信息隐藏？

- ✓模块应该设计得使其所含的信息对那些不需要这些**信息的模块不可访问**；模块间仅仅交换那些为完成系统功能所必需交换的信息

信息隐藏的优点

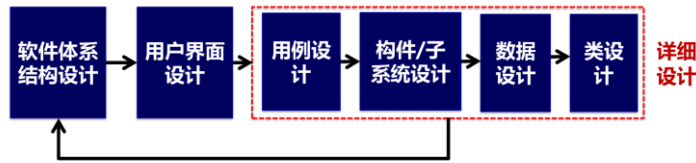
- ✓模块的独立性更好
- ✓支持模块的并行开发（设计和编码）
- ✓减少错误向外传播，便于测试和维护
- ✓便于增加新的功能

功能独立是指软件系统中的每个模块只完成一个相对独立的特定子功能，并且与软件结构中其他模块的接口尽量简单。“高内聚低耦合”

细化是指通过逐步增加细节，将高层抽象的设计模型转化为低层具体实现模型的过程。它是自顶向下（Top-down）设计策略的核心。

重构是在不改变软件外部行为的前提下，通过调整软件内部结构来提高其质量、可理解性及降低修改成本的过程

3. 系统设计从体系结构、数据、接口和构件四方面进行设计。
4. 面向对象的系统设计，各自包含哪些设计内容？



5. 掌握包图、类图和顺序图作法。

第五章 软件实现

1. 软件实现的概念以及工作内容

根据软件**设计模型**，编写出目标软件系统的程序**代码**，并对代码进行必要的**测试**，以发现和纠正代码存在中的**缺陷**，并将可运行的目标代码部署到目标计算机上运行

工作内容

编码 + 单元测试 + 调试

软件实现的其他工作

- 软件架构搭建
 - 软件框架：软件模块组织框架
 - 经过严格验证的通用软件框架是软件实现的集成
 - Spring Boot、Vue.js、React、Django、.NET
- 使用辅助工具
 - 版本管理工具
 - 编辑、编译、部署工具
 - 测试工具、调试工具
 - 集成开发环境
- 软件集成，构建软件系统

2. 程序设计语言的特点和选择

机器语言

- ✓ 由“0”、“1”所组成的机器指令
- ✓ 极为繁琐、费时费力的工作；软件开发效率非常低，而且程序代码可读性非常差，极易出错，不易于维护、移植性差；但程序代码的执行效率会非常高

汇编语言

- ✓ 一种**低级语言**，用助记符代替机器指令的操作码，用地址符号或标号代替指令或操作数的地址
- ✓ 较为低级和复杂，程序可读性差，代码编写的效率低，对代码进行维护非常困难，程序调试也不容易，代码兼容性差
- ✓ 程序代码占用存储空间少、运行速度快、执行效率高

结构化程序设计语言

- ✓以**过程或函数**作为基本的编程单元，采用三类控制结构（顺序、条件和循环）来刻画模块的处理过程和流程
- ✓属于**高级程序设计语言**，程序可读性、可理解性、可维护性等有明显的提升；配套CASE工具较为完善，有结构化程序设计方法学的指导
- ✓不足：以过程和函数作为基本模块，模块的粒度小，可重用性差；程序代码抽象层次低，无法对问题域及其求解进行自然抽象
- ✓如C、Fortran、Pascal等

面向对象程序设计语言

- ✓以类作为基本的模块单元，借助于**面向对象的一组概念和机制**来进行程序设计，
- ✓有系统的**方法学指导**，建立起可直观反映问题域、模块粒度更大、可重用性更好的程序代码，已经成为计算机领域的主流编程语言
- ✓如Java、C++等

描述性程序设计语言

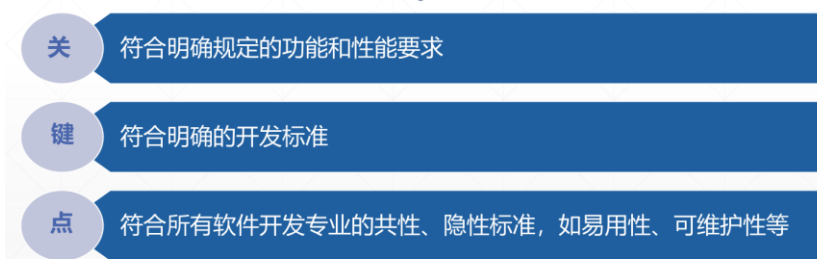
- ✓描述程序需要**解决什么样的问题**，无需在程序中显式地定义如何来解决问题
- ✓如Prolog、Lisp、ML等

3. 理解编程习惯和软件缺陷，以及软件调试的工作

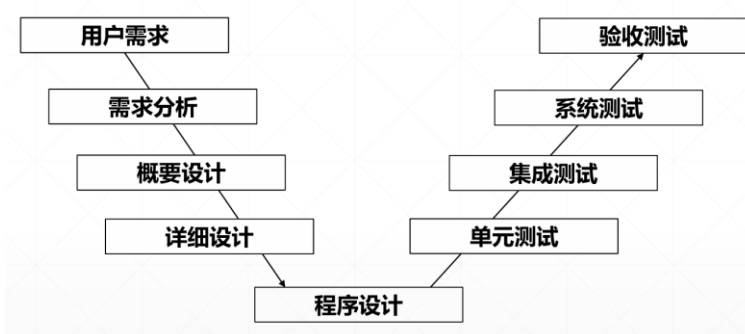
第六章 软件测试（※综合题）

1. 软件质量的概念和关键点

软件质量：明确表示是否符合功能和性能要求，明确地记载开发标准和所有专业开发软件的期望的隐性特点



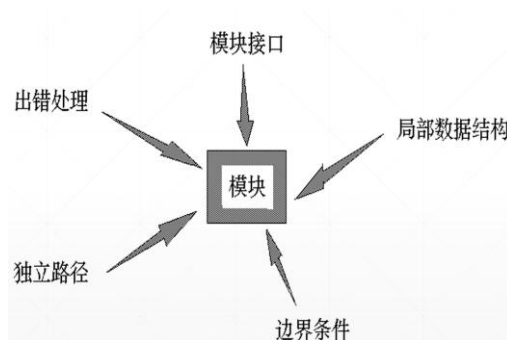
2. 测试策略 V 模型概念，测试与开发的各阶段对应关系



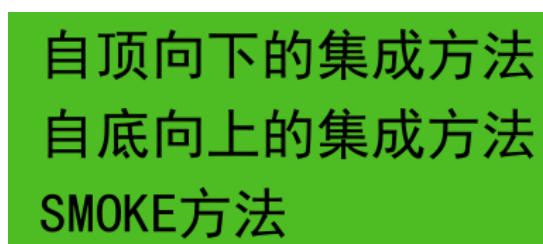
注：左翼为开发阶段，右翼为测试阶段，一一对应。

3. 单元测试的内容、集成测试的分类、系统测试的分类、验收测试的分类

单元测试内容



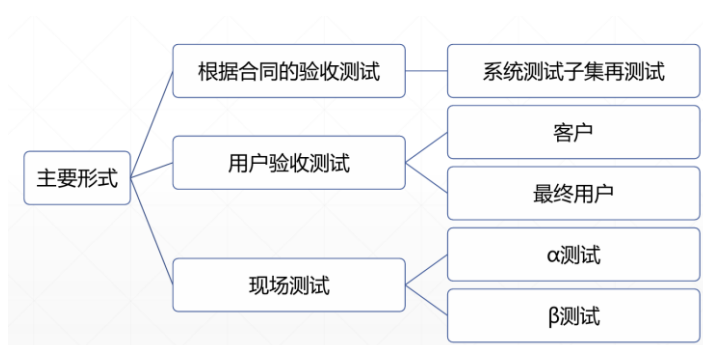
集成测试分类



系统测试分类



验收测试分类



4. 回归测试的概念

回归测试:指有选择地重新测试系统或其组件,以验证对软件的修改没有导致不希望出现的影响,以及系统或组件仍然符合其指定的需求。

5. 测试技术常见术语的概念: 软件缺陷、验证和确认、测试与质量保证、质量与可靠性、调试与测试、测试用例

软件缺陷

软件未实现产品说明书要求的功能。

软件出现了产品说明书指明不能出现的错误。

软件实现了产品说明书未提到的功能。

软件未实现产品说明书虽未明确提及但应该实现的目标。

软件难以理解、不易使用、运行缓慢或者——从测试员的角度看——最终用户会认为不好。

验证 (Verification)

保证软件特定开发阶段的输出已经正确完整地实现了规格说明



确认 (Validation)

对于每个测试级别,都要检查开发活动的输出是否满足具体的需求或与这些特定级别相关的需求

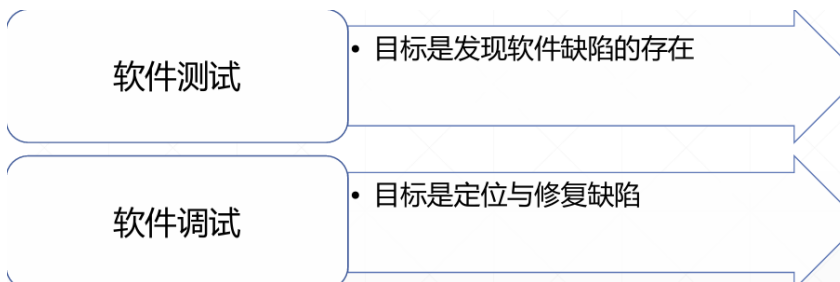


软件测试的定义

在某种指定的条件下对系统或组件操作,观察或记录结果,对系统或组件的某些方面进行评估的过程。

分析软件各项目以检测现有的结果和应有结果之间的差异,并评估软件各项目的特征的过程。

软件质量保证(SQA): 遵照一定的软件生产标准、过程和步骤对软件质量进行评估的活动。



测试用例 (test case) : 是测试输入、执行条件、以及预期结果的集合, 是为特定的目的开发的, 例如执行特定的程序路径或验证与指定的需求相符合。

6. 白盒测试、黑盒测试各有哪些方法

白盒



注意: 语句覆盖又称节点覆盖, 分支覆盖又称边覆盖, 除了上述 4 种, 还有路径覆盖 (全路径覆盖) 和基本路径覆盖 2 种

黑盒



7. 掌握覆盖测试 (语句、分支、条件、条件组合; CFG 的点、边、路径及基本路径覆盖)

\$以题的形式进行复习

作业题 1: 针对如下代码, 画出程序流程图, 活动图, 标记分支。然后选择测试用例, 分别完成语句覆盖、分支覆盖、条件覆盖和条件组合覆盖, 要求针对不同的覆盖, 给出执行过程和对应的测试用例, 以及执行后的输出

```
void func(int a, int b, int c, int d) {  
    if(a>0 && b>100){  
        System.out.println("good");  
    }  
    if(c>200 || d>300){  
        System.out.println("bad");  
        System.out.println("over");  
    }  
}
```

作业题 2: 针对如下代码, 画出控制流图, 用不同方法计算基本路径的数量。然后选择测试用例, 分别完成点覆盖、边覆盖和基本路径覆盖, 要求针对不同的覆盖, 给出执行过程和对应的测试用例, 以及执行后的输出

```
void func(int a, int c) {  
    int i=0;  
    do{  
        if(a>0){  
            System.out.println("good");  
        }  
        if(c>200){  
            System.out.println("bad");  
        }  
        System.out.println("over");  
        i++;  
    } while(i < 2);  
}
```

补充题 1:

某书店针对会员制定了购书优惠逻辑。以下是该功能的伪代码:

```
def calculate_discount(is_member, total_price):  
    if is_member == True and total_price >= 100: # 判定 1  
        if total_price >= 500: # 判定 2  
            discount = 0.8 # 8 折  
        else:  
            discount = 0.9 # 9 折  
        else:  
            discount = 1.0 # 原价  
    return total_price * discount
```

要求：

1. **绘图：**画出该程序的**程序流程图**和**活动图**。
2. **标记：**在流程图上标记出所有的逻辑分支（如 \$L1, L2, \dots\$）。
3. **设计测试用例：**
 - 分别实现**语句覆盖**、**分支覆盖**、**条件覆盖**和**条件组合覆盖**。
 - **格式要求：**请用表格展示，要求针对不同的覆盖，给出执行过程和对应的测试用例，以及执行后的输出

补充题 2：

以下是一个简单的数值处理函数，包含循环和嵌套判断。

```
def summarize_data(n):  
    i = 1  
    total = 0  
    while i <= n:           # 节点 1  
        if i % 2 == 0:      # 节点 2  
            total = total + i # 节点 3  
        else:  
            total = total + 1 # 节点 4  
        i = i + 1           # 节点 5  
    return total           # 节点 6
```

要求：

用不同方法计算基本路径的数量。然后选择测试用例，分别完成点覆盖、边覆盖和基本路径覆盖，要求针对不同的覆盖，给出执行过程和对应的测试用例，以及执行后的输出

8. 等价类划分测试方法有效等价类和无效等价类划分，对应测试用例的设计？

\$以题的形式进行复习

作业题：某保险公司系统在的某个产品在输入被保险人数据时，需要输入年龄，体重，类型为整数型，年龄范围：45 岁至 59 岁（ $45 \leq \text{age} \leq 59$ ），体重范围：55 到 70 公斤（ $55 \leq \text{weight} \leq 70$ ），请结合等价类划分、边界值分析法设计测试用例。

要求：用表格给出等价类划分（给出有效和无效等价类）。

针对体重，指出边界开闭区间上点、内点和离点，进行边界值分析合理设计用例，分别用表格给出最终的测试用例

补充题：

某银行开发的“青少年储蓄账户”申请系统，在输入申请人信息时有如下限制：

1. **开户人年龄 (Age)：**必须是整数，范围在 10 岁至 18 岁之间（含 10 岁和 18 岁）。
2. **起存金额 (Amount)：**必须是整数，最低起存 100 元，最高限额 5000 元（含 100 和 5000）。

要求:

1. **等价类划分:** 请列出《等价类划分表》, 包括输入条件、有效等价类(编号)和无效等价类(编号)。
 2. **边界值分析:**
 - 针对**起存金额 (Amount)**, 明确指出其边界的开区间、闭区间。
 - 找出边界上的上点、内点和离点。
 3. **测试用例表:**
 - 综合以上方法, 合理设计最终的测试用例。
 - **表格格式:** 用例编号、输入项(Age)、输入项(Amount)、测试目的、预期结果。
9. 边界测试方法(开闭区间、上点、离点、内点等等并设计测试用例)
\$以题的形式进行复习
见第 8 点

第八章 项目管理

1. 软件项目管理的概念, 软件项目管理的对象

概念

对软件项目所涉及的过程、人员、产品、成本和进度等要素进行度量、分析、规划、组织和控制的过程, 以确保软件项目按照预定的成本、进度、质量要求顺利完成

对象



2. 软件度量、测量和估算的概念和方法, 软件项目计划;

度量

软件度量(Metrics)是指对软件产品、软件开发过程或者资源的简单属性的定量描述

测量

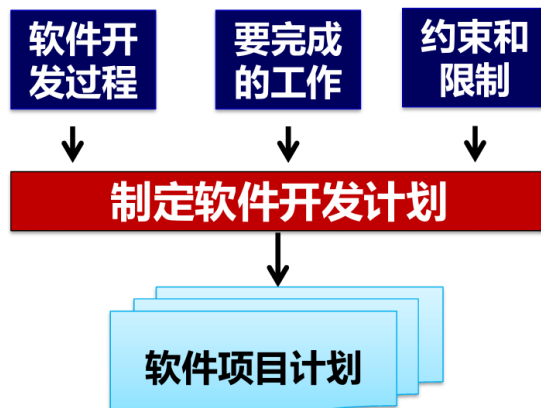
对软件产品、软件开发过程和资源**复杂属性的定量描述**, 它是简单属性度量值函数, **用于事后或实时状态**, 如软件可靠性

估算

对软件产品、软件开发过程和资源**复杂属性的定量描述**, 它是简单属性度量值的函数, 软件估算**用于事前**

软件项目计划

✓对软件项目实施所涉及的活动、人员的安排、任务的划分、开发进度、资源的分配和使用等方面作出的**预先规划**



3. 软件配置管理，软件风险管理，软件质量保证（参见测试那一章第一部分）

什么是软件配置管理？

□在软件生命周期中对SCI进行的以下工作

- ✓系统地控制SCI的**标识、存储、更动和发放**
- ✓**记录、报告其状态**
- ✓验证SCI的**正确性和一致性**
- ✓对上述工作的**审计**

风险管理的组成

